

西南科技大学计算机科学与技术学院

实验报告

实验名称 Linux 环境安装配置

实验地点 东 11C225 大学计算机实验室

实验日期 2024/09/26

指导教师 韦勇

学生班级

学生姓名

学生学号

提交日期 2024/09/30

一、实验目的

了解 Linux 上软件安装的不同方式,验证在 Linux 字符界面中安装和卸载二进制包及源代码的过程,掌握 Linux 字符界面中安装和卸载软件的方法。

Linux 上软件安装的不同方式,验证在 Linux 上通过网络安装软件的方法。

通过案例系统的安装,掌握 Linux 字符界面本地安装软件的办法。

二、实验设计

实验一: 点击桌面图标为 “Xfce 终端”, 打开 Terminal, 在 Linux 下使用 tar.gz 包进行本地安装软件, 并通过文档更改安装路径。

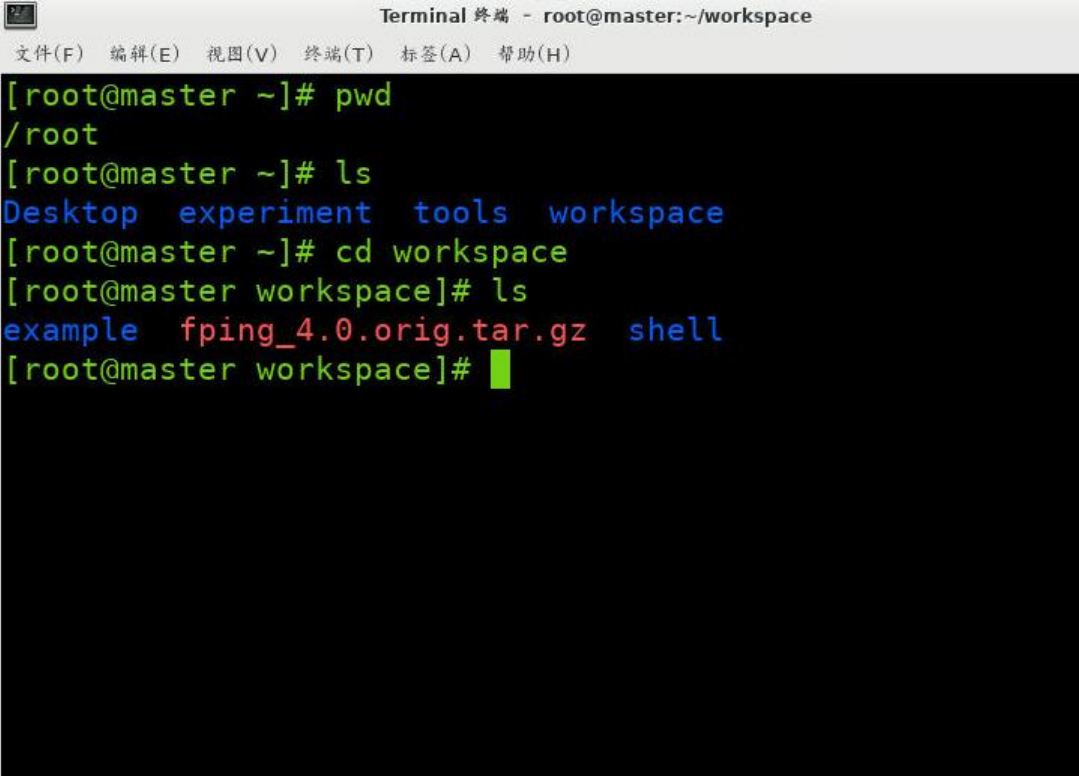
实验二: 点击桌面图标为 “Xfce 终端”, 打开 Terminal, 配置互联网源后进行 yum 远程安装 nmap。

实验三: 点击桌面图标为 “Xfce 终端”, 打开 Terminal, 先利用 yum 远程安装 Linuxer 客户端的依赖项 jupyter notebook, 再利用 tar.gz 文件包本地安装 wput, 最后完成 Linuxer 客户端安装。

三、实验记录

实验一：本地安装

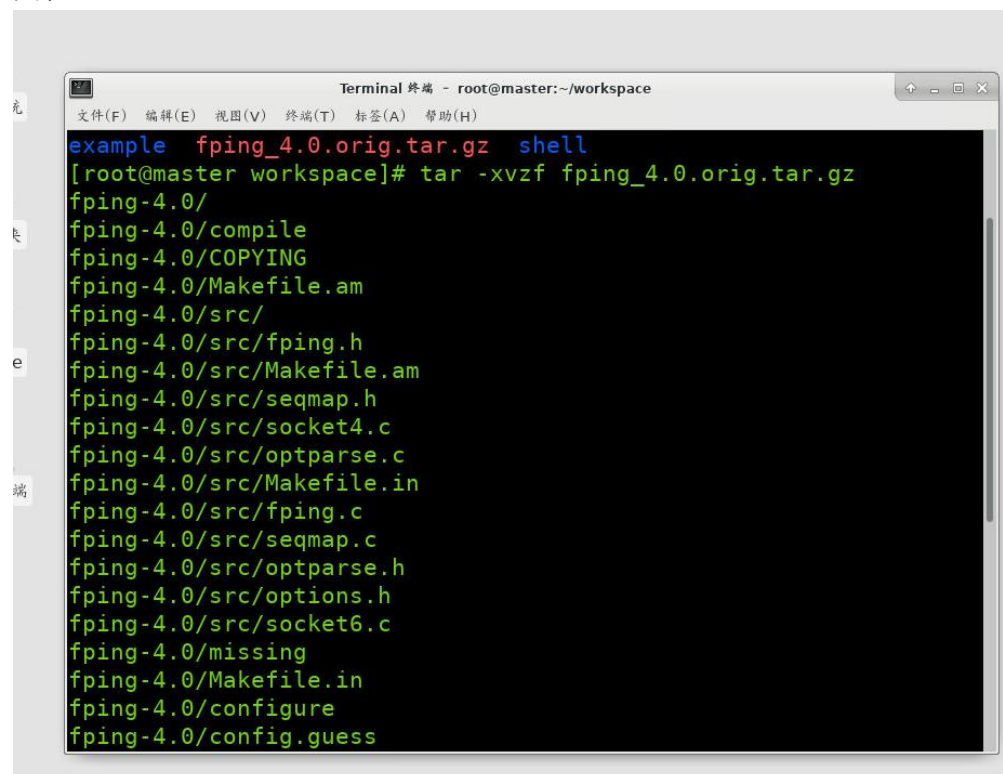
查看 tar.gz 中的 INSTALL 文件, 学习如何设置安装路径, 并将 fping 安装到 /opt 目录下。

A screenshot of a Linux terminal window titled "Terminal 终端 - root@master: ~/workspace". The terminal shows the following commands and output:

```
[root@master ~]# pwd
/root
[root@master ~]# ls
Desktop  experiment  tools  workspace
[root@master ~]# cd workspace
[root@master workspace]# ls
example  fping_4.0.orig.tar.gz  shell
[root@master workspace]#
```

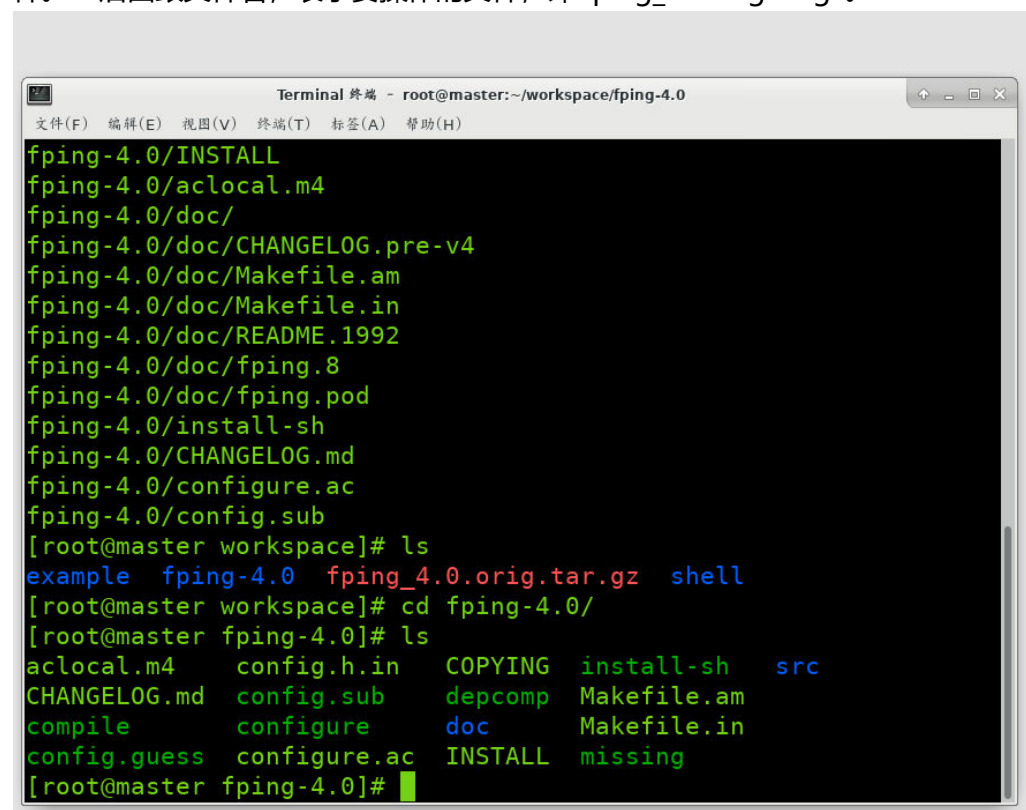
切换到 workspace 目录下, 查看当前文件夹所含文件, 发现有 fping_4.0.orig.tar.gz

安装包。



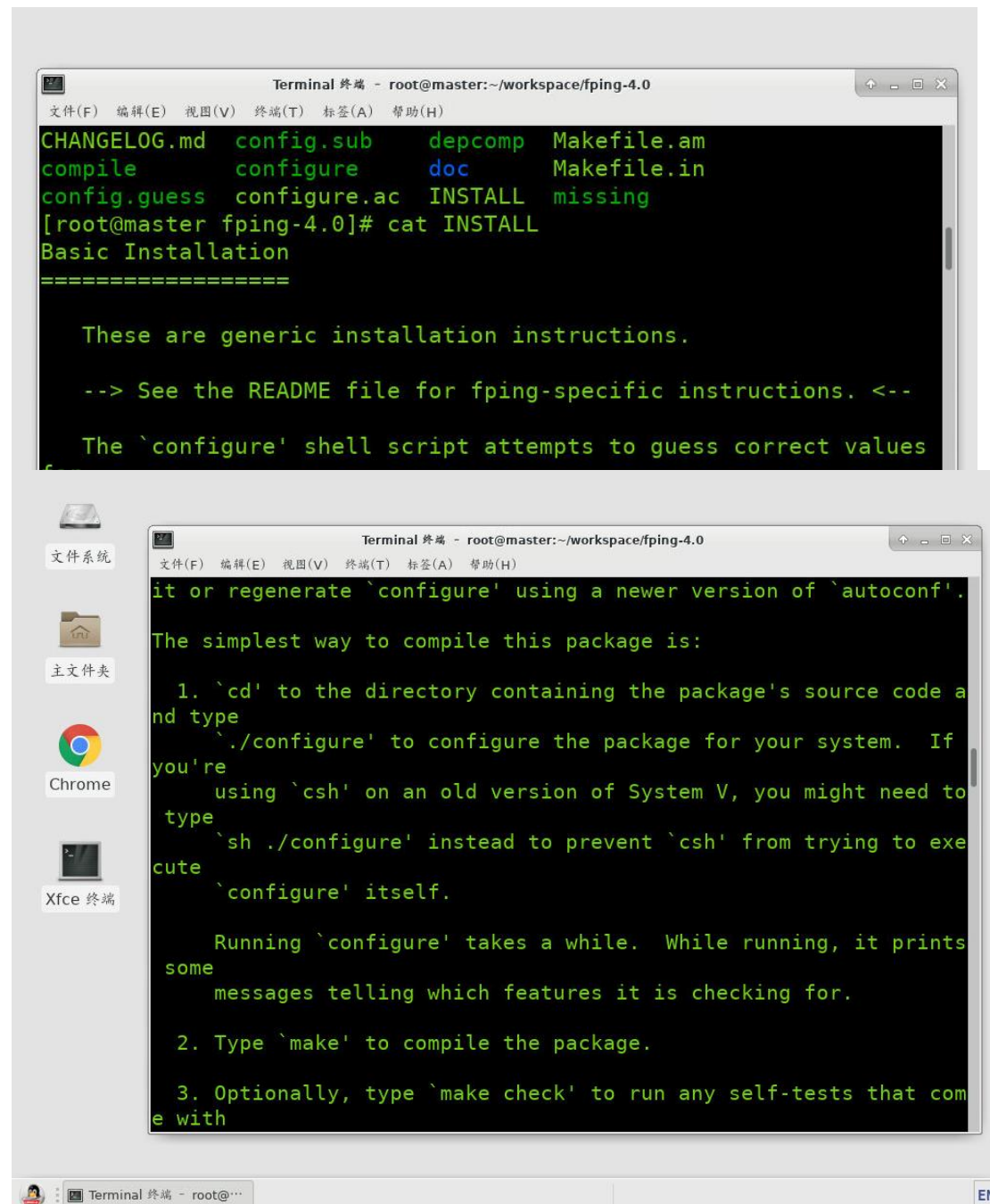
```
Terminal 终端 - root@master:~/workspace
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)
example fping_4.0.orig.tar.gz shell
[root@master workspace]# tar -xvzf fping_4.0.orig.tar.gz
fping-4.0/
fping-4.0/compile
fping-4.0/COPYING
fping-4.0/Makefile.am
fping-4.0/src/
fping-4.0/src/fping.h
fping-4.0/src/Makefile.am
fping-4.0/src/seqmap.h
fping-4.0/src/socket4.c
fping-4.0/src/optparse.c
fping-4.0/src/Makefile.in
fping-4.0/src/fping.c
fping-4.0/src/seqmap.c
fping-4.0/src/optparse.h
fping-4.0/src/options.h
fping-4.0/src/socket6.c
fping-4.0/missing
fping-4.0/Makefile.in
fping-4.0/configure
fping-4.0/config.guess
```

使用 `tar -xvzf fping_4.0.orig.tar.gz` 进行解压。tar 是用来打包和压缩文件的命令行工具。-x 告诉 `tar` 要执行解压缩 (extract) 操作。-v 代表 verbose (详细)，即在解压缩的过程中显示文件列表，让我们知道哪些文件被解压了。-z 则是使用 gzip 程序来解压缩文件。-f 后面跟文件名，表示要操作的文件，即 `fping_4.0.orig.tar.gz`。



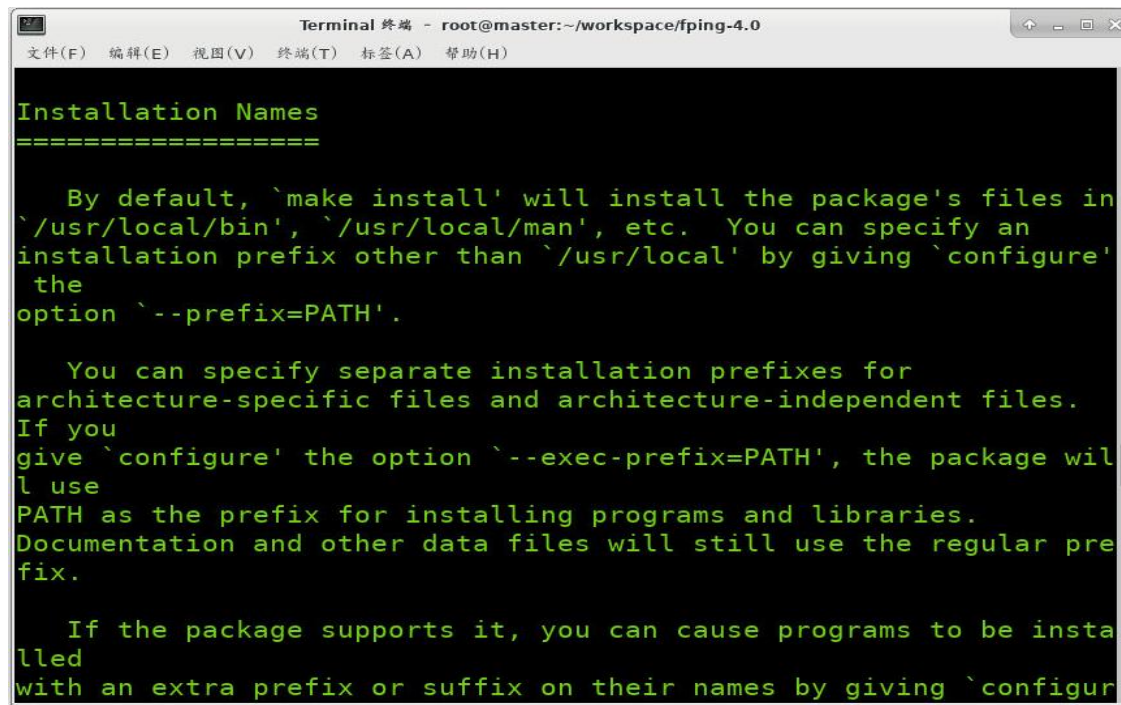
```
Terminal 终端 - root@master:~/workspace/fping-4.0
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)
fping-4.0/INSTALL
fping-4.0/aclocal.m4
fping-4.0/doc/
fping-4.0/doc/CHANGELOG.pre-v4
fping-4.0/doc/Makefile.am
fping-4.0/doc/Makefile.in
fping-4.0/doc/README.1992
fping-4.0/doc/fping.8
fping-4.0/doc/fping.pod
fping-4.0/install-sh
fping-4.0/CHANGELOG.md
fping-4.0/configure.ac
fping-4.0/config.sub
[root@master workspace]# ls
example fping-4.0 fping_4.0.orig.tar.gz shell
[root@master workspace]# cd fping-4.0/
[root@master fping-4.0]# ls
aclocal.m4    config.h.in  COPYING     install-sh   src
CHANGELOG.md  config.sub   depcomp     Makefile.am
compile       configure    doc         Makefile.in
config.guess  configure.ac INSTALL      missing
[root@master fping-4.0]#
```

解压后发现当前目录下多了一个 fping_4.0 文件夹，进入 fping_4.0，使用 ls 查看当前目录下文件，发现存在 INSTALL 文件。tar.gz 是一个标准的压缩包，需使用编译命令完成编译，然后完成软件的安装、卸载操作。tar.gz 的安装流程一般遵循《GNU 编码标》，不过也存在软件开发者自己制定的情况，可以参考安装包中的 README 文档，这里即 INSTALL 文件，查看 INSTALL 文件内容，寻找安装路径的更改方法。



编译这个包最简单的方法是：

1. cd 到包含软件包的源代码的目录，并键入 ./configure 为您的系统配置软件包。运行 configure 需要一段时间。在运行时，它会打印一些消息，说明它正在检查哪些功能。
2. 键入 " make " 以编译程序包。
3. (可选) 键入 " make check " 以运行出现的任何自检。

A terminal window titled "Terminal 终端 - root@master:~/workspace/fping-4.0" with a menu bar containing "文件(F)", "编辑(E)", "视图(V)", "终端(T)", "标签(A)", and "帮助(H)". The terminal output shows the "Installation Names" section, which explains that by default, 'make install' will install files in '/usr/local/bin', '/usr/local/man', etc. It also mentions that you can specify an installation prefix other than '/usr/local' by giving 'configure' the option '--prefix=PATH'. Further, it states that you can specify separate installation prefixes for architecture-specific files and architecture-independent files by giving 'configure' the option '--exec-prefix=PATH'. Finally, it notes that if the package supports it, you can cause programs to be installed with an extra prefix or suffix on their names by giving 'configure' the option '--program-prefix=PREFIX' or '--program-suffix=SUFFIX'.

```
Installation Names
=====

By default, `make install' will install the package's files in
`/usr/local/bin', `/usr/local/man', etc. You can specify an
installation prefix other than `/usr/local' by giving `configure'
the
option `--prefix=PATH'.

You can specify separate installation prefixes for
architecture-specific files and architecture-independent files.
If you
give `configure' the option `--exec-prefix=PATH', the package will
use
PATH as the prefix for installing programs and libraries.
Documentation and other data files will still use the regular pre
fix.

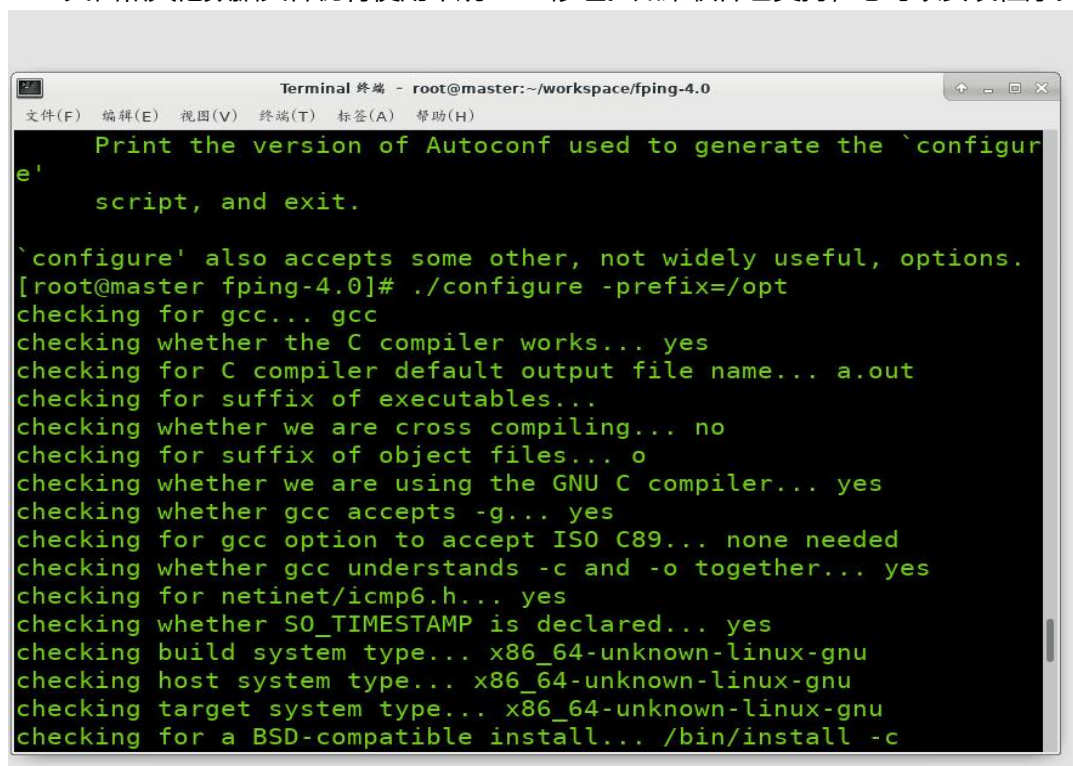
If the package supports it, you can cause programs to be insta
lled
with an extra prefix or suffix on their names by giving `configur
```

安装名称

默认情况下, " make install " 将在 "/usr/local/bin "、"/usr/local/man " 等目录中安装软件包的文件。您可以通过为 " configure " 指定 "/usr/local " 以外的安装前缀选项 "--prefix=path "。

您可以为指定单独的安装前缀特定于体系结构的文件和独立于体系结构的文件。如果为 " configure " 提供 "--exec-prefix=path " 选项, 则软件包将使用 path 作为安装程序和库的前缀。

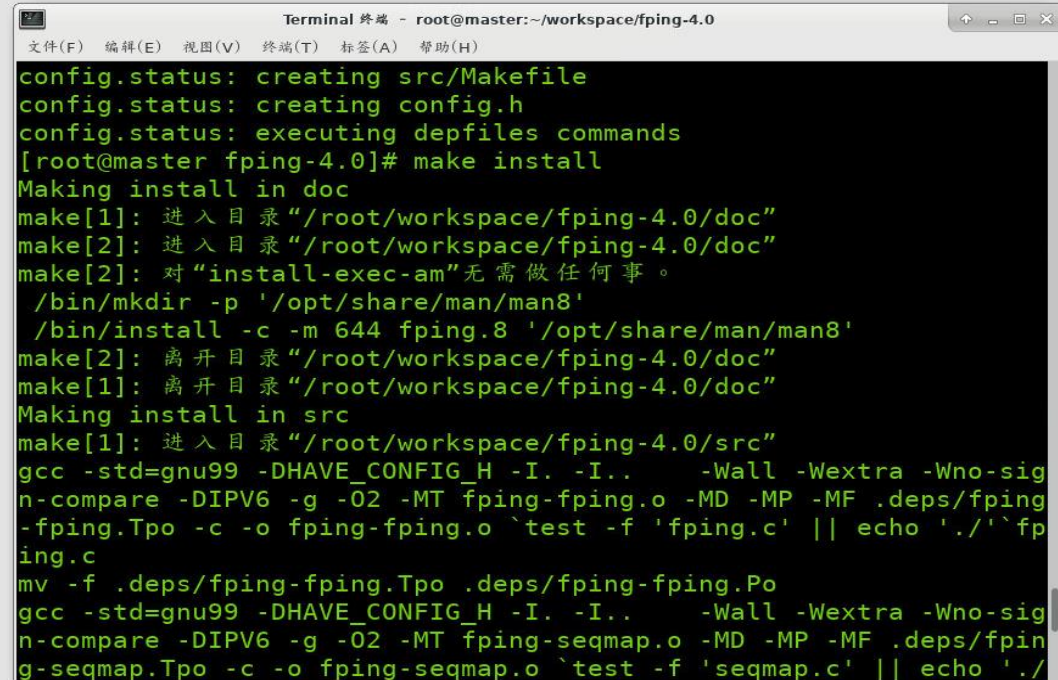
文档和其他数据文件仍将使用常规 PRE 修理。如果软件包支持, 您可以安装程序。

A terminal window titled "Terminal 终端 - root@master:~/workspace/fping-4.0" with a menu bar containing "文件(F)", "编辑(E)", "视图(V)", "终端(T)", "标签(A)", and "帮助(H)". The terminal output shows the output of the 'configure' script. It starts with the instruction to print the version of Autoconf used to generate the 'configure' script and exit. Then, it shows the command '[root@master fping-4.0]# ./configure -prefix=/opt' and the subsequent checks for gcc, C compiler, and various system features. The checks for gcc, C compiler, and various system features are all successful, and the build system type is identified as 'x86_64-unknown-linux-gnu'.

```
Print the version of Autoconf used to generate the `configure'
script, and exit.

`configure' also accepts some other, not widely useful, options.
[root@master fping-4.0]# ./configure -prefix=/opt
checking for gcc... gcc
checking whether the C compiler works... yes
checking for C compiler default output file name... a.out
checking for suffix of executables...
checking whether we are cross compiling... no
checking for suffix of object files... o
checking whether we are using the GNU C compiler... yes
checking whether gcc accepts -g... yes
checking for gcc option to accept ISO C89... none needed
checking whether gcc understands -c and -o together... yes
checking for netinet/icmp6.h... yes
checking whether SO_TIMESTAMP is declared... yes
checking build system type... x86_64-unknown-linux-gnu
checking host system type... x86_64-unknown-linux-gnu
checking target system type... x86_64-unknown-linux-gnu
checking for a BSD-compatible install... /bin/install -c
```

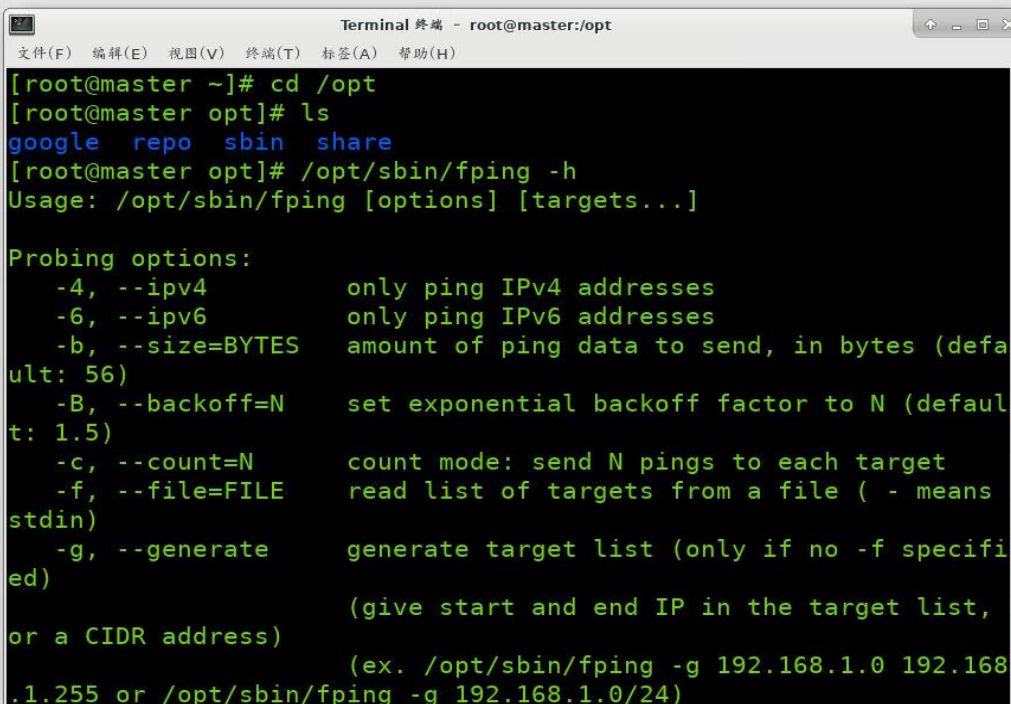
根据以上文档内容，输入 “./configure -prefix=/opt” 更改安装路径并配置软件包。
./configure 是一个常见的脚本，用于配置软件以适应不同的系统环境。这个脚本通常在软件的源代码编译之前运行，以检查系统环境并设置编译参数。



```
Terminal 终端 - root@master:~/workspace/fping-4.0
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)
config.status: creating src/Makefile
config.status: creating config.h
config.status: executing depfiles commands
[root@master fping-4.0]# make install
Making install in doc
make[1]: 进入目录 "/root/workspace/fping-4.0/doc"
make[2]: 进入目录 "/root/workspace/fping-4.0/doc"
make[2]: 对 "install-exec-am" 无需做任何事。
/bin/mkdir -p '/opt/share/man/man8'
/bin/install -c -m 644 fping.8 '/opt/share/man/man8'
make[2]: 离开目录 "/root/workspace/fping-4.0/doc"
make[1]: 离开目录 "/root/workspace/fping-4.0/doc"
Making install in src
make[1]: 进入目录 "/root/workspace/fping-4.0/src"
gcc -std=gnu99 -DHAVE_CONFIG_H -I. -I.. -Wall -Wextra -Wno-sign-compare -DIPV6 -g -O2 -MT fping-fping.o -MD -MP -MF .deps/fping-fping.Tpo -c -o fping-fping.o `test -f 'fping.c' || echo './'`fping.c
mv -f .deps/fping-fping.Tpo .deps/fping-fping.Po
gcc -std=gnu99 -DHAVE_CONFIG_H -I. -I.. -Wall -Wextra -Wno-sign-compare -DIPV6 -g -O2 -MT fping-seqmap.o -MD -MP -MF .deps/fping-seqmap.Tpo -c -o fping-seqmap.o `test -f 'seqmap.c' || echo './'`seqmap.c
```

再使用 make install 进行安装。

make install 是一个用于编译和安装软件的命令，它通常在软件的源代码目录中执行，是在软件配置（通过 ./configure 脚本）之后，编译（通过 make 命令）之后执行的步骤。



```
Terminal 终端 - root@master:/opt
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)
[root@master ~]# cd /opt
[root@master opt]# ls
google repo sbin share
[root@master opt]# /opt/sbin/fping -h
Usage: /opt/sbin/fping [options] [targets...]

Probing options:
  -4, --ipv4           only ping IPv4 addresses
  -6, --ipv6           only ping IPv6 addresses
  -b, --size=BYTES     amount of ping data to send, in bytes (default: 56)
  -B, --backoff=N      set exponential backoff factor to N (default: 1.5)
  -c, --count=N        count mode: send N pings to each target
  -f, --file=FILE      read list of targets from a file ( - means stdin)
  -g, --generate       generate target list (only if no -f specified)
                       (give start and end IP in the target list, or a CIDR address)
                       (ex. /opt/sbin/fping -g 192.168.1.0 192.168.1.255 or /opt/sbin/fping -g 192.168.1.0/24)
```

如果安装成功，则输入 `/opt/sbin/fping -h` 有信息返回。

因为执行 `make install` 命令安装软件时，软件的可执行文件会被复制到系统的标准目录中，这些目录通常包括 `/bin`、`/sbin`、`/usr/bin`、`/usr/sbin` 等。因为我们使用 `prefix=/opt` 选项指定了安装前缀，所有 `fping` 可执行文件会被安装到 `/opt` 下的相应子目录中即 `/opt/sbin`。`make install` 命令将 `fping` 可执行文件安装到了 `/opt/sbin` 目录下。

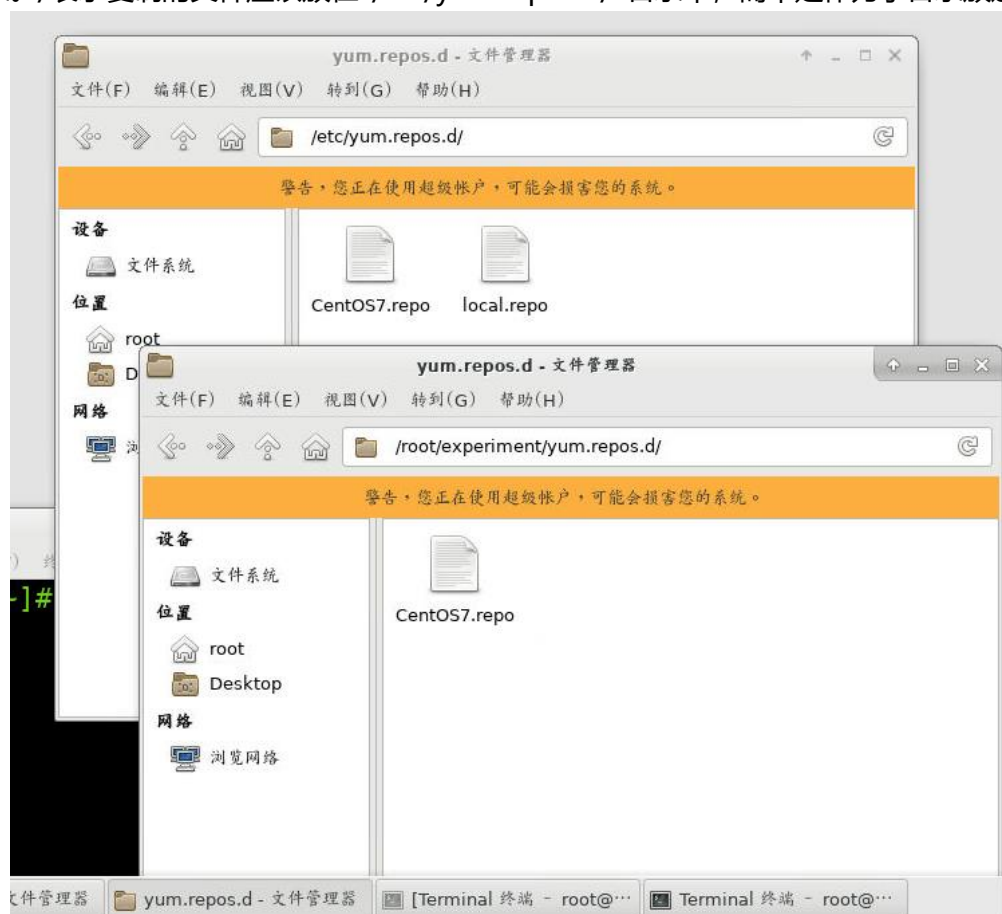
实验二：远程安装

为 `yum` 添加互联网上的源，并安装 `nmap` 软件。（提示：拷贝 `/root/experiment/yum.repos.d/` 中的源配置文件到 `/etc/yum.repos.d/` 中）

```
[root@master ~]# cp /root/experiment/yum.repos.d/* /etc/yum.repos.d/
[root@master ~]# ls /etc/yum.repos.d/
CentOS7.repo  local.repo
[root@master ~]#
```

利用 `cp` 命令，将 `/root/experiment/yum.repos.d/` 中的源配置文件拷贝到 `/etc/yum.repos.d/` 中，即 `cp /root/experiment/yum.repos.d/* /etc/yum.repos.d/`

*是一个通配符，表示匹配 `/root/experiment/yum.repos.d/` 目录下的所有文件和子目录。`/` 表示复制的文件应该放在 `/etc/yum.repos.d/` 目录中，而不是作为子目录放进去。



如果目标目录中已经存在与源目录中同名的文件，那么默认情况下会覆盖它们，除非使用了 `-i` (交互式) 选项，这样在覆盖前会提示我们确认。

如果目标目录中没有足够的空间来存放复制的文件，命令会报错并停止复制。

```
Terminal 终端 - root@master:~
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)
错误：无须任何处理
[root@master ~]# yum install nmap -y
已加载插件：ovl, ulninfo
源 'local' 在配置文件中未指定名字，使用标识代替
正在解决依赖关系
--> 正在检查事务
--> 软件包 nmap.x86_64.2.6.40-19.el7 将被 安装
--> 正在处理依赖关系 nmap-ncat = 2:6.40-19.el7，它被软件包 2:nmap-6.40-19.el7.x86_64 需要
--> 正在处理依赖关系 libpcap.so.1()(64bit)，它被软件包 2:nmap-6.40-19.el7.x86_64 需要
--> 正在检查事务
--> 软件包 libpcap.x86_64.14.1.5.3-13.el7_9 将被 安装
--> 软件包 nmap-ncat.x86_64.2.6.40-19.el7 将被 安装
--> 解决依赖关系完成

依赖关系解决

=====
Package          架构      版本                源                大小
=====
正在安装：
```

使用 `yum install nmap -y` 进行 nmap 的安装。

```
Terminal 终端 - root@master:~
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)
-----
总计                               9.5 MB/s | 4.3 MB  00:00
Running transaction check
Running transaction test
Transaction test succeeded
Running transaction
 正在安装      : 14:libpcap-1.5.3-13.el7_9.x86_64      1/3
 正在安装      : 2:nmap-ncat-6.40-19.el7.x86_64       2/3
 正在安装      : 2:nmap-6.40-19.el7.x86_64            3/3
 验证中        : 2:nmap-ncat-6.40-19.el7.x86_64       1/3
 验证中        : 14:libpcap-1.5.3-13.el7_9.x86_64    2/3
 验证中        : 2:nmap-6.40-19.el7.x86_64            3/3

已安装：
nmap.x86_64 2:6.40-19.el7

作为依赖被安装：
libpcap.x86_64 14:1.5.3-13.el7_9
nmap-ncat.x86_64 2:6.40-19.el7

完毕！
[root@master ~]#
```

yum 是 Linux 系统中一个流行的包管理器，主要用于基于 Red Hat 的发行版，如 Red Hat Enterprise Linux (RHEL)、CentOS 和 Fedora。它用于安装、更新、删除和管

理软件包。

install: 这是 yum 命令的动作之一，用于安装新的软件包。

nmap: 这是要安装的软件包的名称。nmap 是一个开源的网络扫描工具，用于网络发现和安全审计。

-y: 这是一个选项，表示在安装过程中自动回答所有提示为“是” (yes)。

实验三：Linuxer 客户端安装

Jupyter notebook 安装:

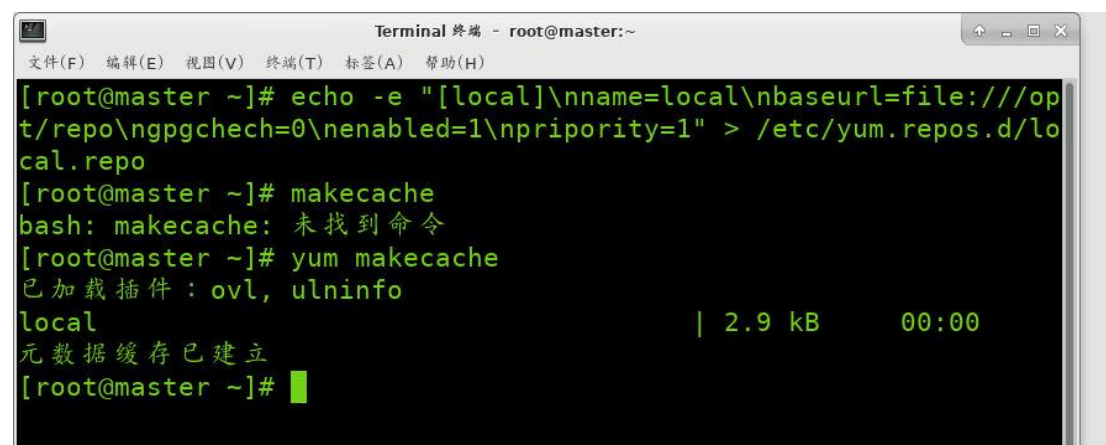
```
[root@master ~]# echo -e "[local]\nname=local\nbaseurl=file:///opt/repo\nngpgcheck=0\nenabled=1\npriority=1" > /etc/yum.repos.d/local.repo
```

使用 yum 远程安装前需要先配置互联网源，因此修改 /etc/yum.repos.d/local.repo 配置文件。

echo 用于将文本输出到标准输出的命令，-e 用于开启解释转义字符，[local]\nname=local\nbaseurl=file:///opt/repo\nngpgcheck=0\nenabled=1\npriority=1: 这是要输出的文本内容，包含仓库的配置信息：

定义了一个名为 `local` 的仓库并且禁用了 GPG 校验，GPG 校验用于验证软件包的签名，设置为 0 表示不进行校验。enabled=1 启用了这个仓库，`1` 表示仓库是可用的，priority=1 设置了仓库的优先级，这里设置为 `1`。

> 这是重定向操作符，用于将 echo 命令的输出即 [local]\nname=local\nbaseurl=file:///opt/repo\nngpgcheck=0\nenabled=1\npriority=1:重定向到文件中，如果文件不存在会被创建。



```
Terminal 终端 - root@master:~
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)

[root@master ~]# echo -e "[local]\nname=local\nbaseurl=file:///opt/repo\nngpgcheck=0\nenabled=1\npriority=1" > /etc/yum.repos.d/local.repo
[root@master ~]# makecache
bash: makecache: 未找到命令
[root@master ~]# yum makecache
已加载插件：ovl, ulninfo
local                               | 2.9 kB    00:00
元数据缓存已建立
[root@master ~]#
```

yum makecache 命令是 YUM 包管理器的一个实用工具，用于创建一个软件包元数据的本地缓存。这个缓存可以加快后续的 YUM 命令执行速度，因为它减少了从远程仓库下载信息的次数，可以确保本地缓存与远程仓库保持同步，从而获取最新的软件包信息。

```
[root@master ~]# yum install python3-pip
已加载插件：ovl, ulninfo
local | 2.9 kB | 00:00
正在解决依赖关系
--> 正在检查事务
---> 软件包 python3-pip.noarch.0.9.0.3-8.el7 将被 安装
--> 正在处理依赖关系 python(abi) = 3.6，它被软件包 python3-pip-9
0.3-8.el7.noarch 需要
--> 正在处理依赖关系 python3-setuptools，它被软件包 python3-pip-9
.0.3-8.el7.noarch 需要
--> 正在处理依赖关系 /usr/bin/python3，它被软件包 python3-pip-9.
.3-8.el7.noarch 需要
--> 正在检查事务
---> 软件包 python3.x86_64.0.3.6.8-18.0.3.el7 将被 安装
--> 正在处理依赖关系 python3-libs(x86-64) = 3.6.8-18.0.3.el7，它
```

因为 jupyter notebook 是 Python3 语言开发的 Web 应用程序，需使用 pip 工具进行，为此，需 python3 和 python3-pip 包，使用 yum install python3-pip 安装 python3-pip。

```
[root@master ~]# pip3 install pip --upgrade
WARNING: Running pip install with root privileges is generally not
a good idea. Try `pip3 install --user` instead.
Collecting pip
  Downloading https://files.pythonhosted.org/packages/a4/6d/6463
49a933f547439d6b5b98b46af8742cc03ae83543e4d7688c2420f8b/pip-21.3
1-py3-none-any.whl (1.7MB)
    0% | | 10kB 1.6MB/s eta 0:00:
    1% | | 20kB 556kB/s eta 0:00:
    1% | | 30kB 820kB/s eta 0:00:
    2% | | 40kB 337kB/s eta 0:00:
    2% | | 51kB 370kB/s eta 0:00:
```

pip3 install -i https://pypi.tuna.tsinghua.edu.cn/simple pip --upgrade 的作用是：使用清华大学提供的 Python 包镜像源来升级 pip 工具到最新版本。

pip3 是 Python 3 的包管理工具，用于安装和管理 Python 包。

install 是 pip 命令的一个参数，表示要安装一个包。

-i 是 --index-url 的缩写，用于指定一个自定义的 Python 包索引源 (repository)。在安装包时，pip 会从这个指定的源下载包。

https://pypi.tuna.tsinghua.edu.cn/simple 是清华大学提供的 Python 包镜像源地址。使用这个源可以加速下载 Python 包，因为它提供了一个地理位置更靠近中国大陆用户的镜像服务。

--upgrade 是 pip 命令的一个参数，用于升级包到最新版本。

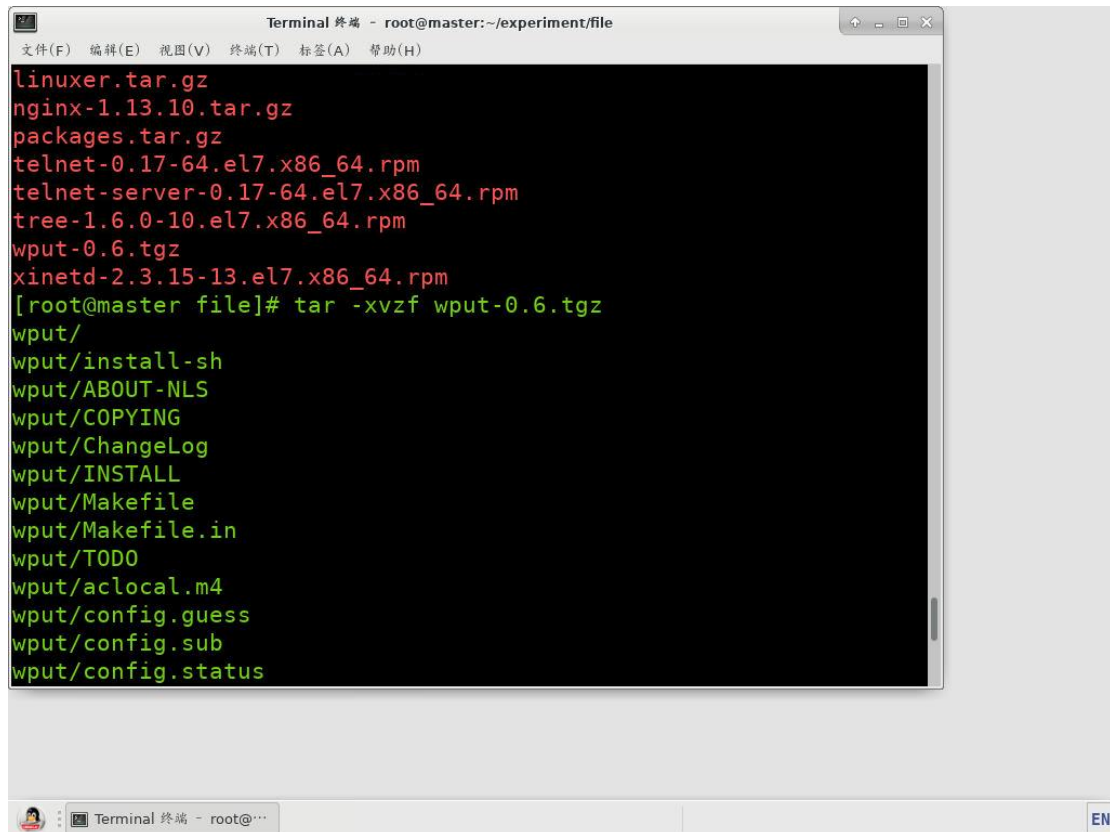
```
Terminal 终端 - root@master:~/experiment/file
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)
trypoints-0.4 idna-3.10 immutables-0.19 importlib-metadata-4.8.3
ipykernel-5.5.6 ipython-7.16.3 ipython-genutils-0.2.0 ipywidgets-
7.8.4 jedi-0.17.2 jinja2-3.0.3 json5-0.9.16 jsonschema-3.2.0 jupy
ter-1.1.1 jupyter-client-7.1.2 jupyter-console-6.4.3 jupyter-core
-4.9.2 jupyter-server-1.13.1 jupyterlab-3.2.9 jupyterlab-pygments
-0.1.2 jupyterlab-server-2.10.3 jupyterlab-widgets-1.1.10 mistune
-0.8.4 nbclassic-0.3.5 nbclient-0.5.9 nbconvert-6.0.7 nbformat-5.
1.3 nest-asyncio-1.6.0 notebook-6.4.10 packaging-21.3 pandocfilte
rs-1.5.1 parso-0.7.1 pexpect-4.9.0 pickleshare-0.7.5 prometheus-c
lient-0.17.1 prompt-toolkit-3.0.36 ptyprocess-0.7.0 pycparser-2.2
1 pygments-2.14.0 pyparsing-3.1.4 pyrsistent-0.18.0 python-dateut
il-2.9.0.post0 pytz-2024.2 pyzmq-25.1.2 requests-2.27.1 six-1.16.
0 sniffio-1.2.0 terminado-0.12.1 testpath-0.6.0 tornado-6.1 trait
lets-4.3.3 typing-extensions-4.1.1 urllib3-1.26.20 wcwidth-0.2.13
webencodings-0.5.1 websocket-client-1.3.1 widgetsnbextension-3.6
.9 zipp-3.6.0
WARNING: Running pip as the 'root' user can result in broken perm
issions and conflicting behaviour with the system package manager
. It is recommended to use a virtual environment instead: https://
/pip.pypa.io/warnings/venv
```

安装 jupyter notebook, Jupyter Notebook 是一个开源的 Web 应用程序, 允许你创建和共享包含实时代码、方程、可视化和解释性文本的文档。它广泛用于数据分析、机器学习、科学计算和许多其他领域。

```
Terminal 终端 - root@master:~/experiment/file
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)
0 sniffio-1.2.0 terminado-0.12.1 testpath-0.6.0 tornado-6.1 trait
lets-4.3.3 typing-extensions-4.1.1 urllib3-1.26.20 wcwidth-0.2.13
webencodings-0.5.1 websocket-client-1.3.1 widgetsnbextension-3.6
.9 zipp-3.6.0
WARNING: Running pip as the 'root' user can result in broken perm
issions and conflicting behaviour with the system package manager
. It is recommended to use a virtual environment instead: https://
/pip.pypa.io/warnings/venv
[root@master ~]# pip3 install -i https://pypi.tuna.tsinghua.edu.c
n/simple bash_kernel
WARNING: pip is being invoked by an old script wrapper. This will
fail in a future version of pip.
Please see https://github.com/pypa/pip/issues/5599 for advice on
fixing the underlying issue.
To avoid this problem you can invoke Python with '-m pip' instead
of running pip directly.
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Collecting bash_kernel
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/09/f0/c2
3032756e91cc04a88a30ccb6c2d7a4d742b39712b7f0e3edaaa772a7cf/bash_k
ernel-0.9.3-py2.py3-none-any.whl (617 kB)
    | 10 kB 13.2 MB/s eta 0:00:
    | 20 kB 13.2 MB/s eta 0:00:0
```

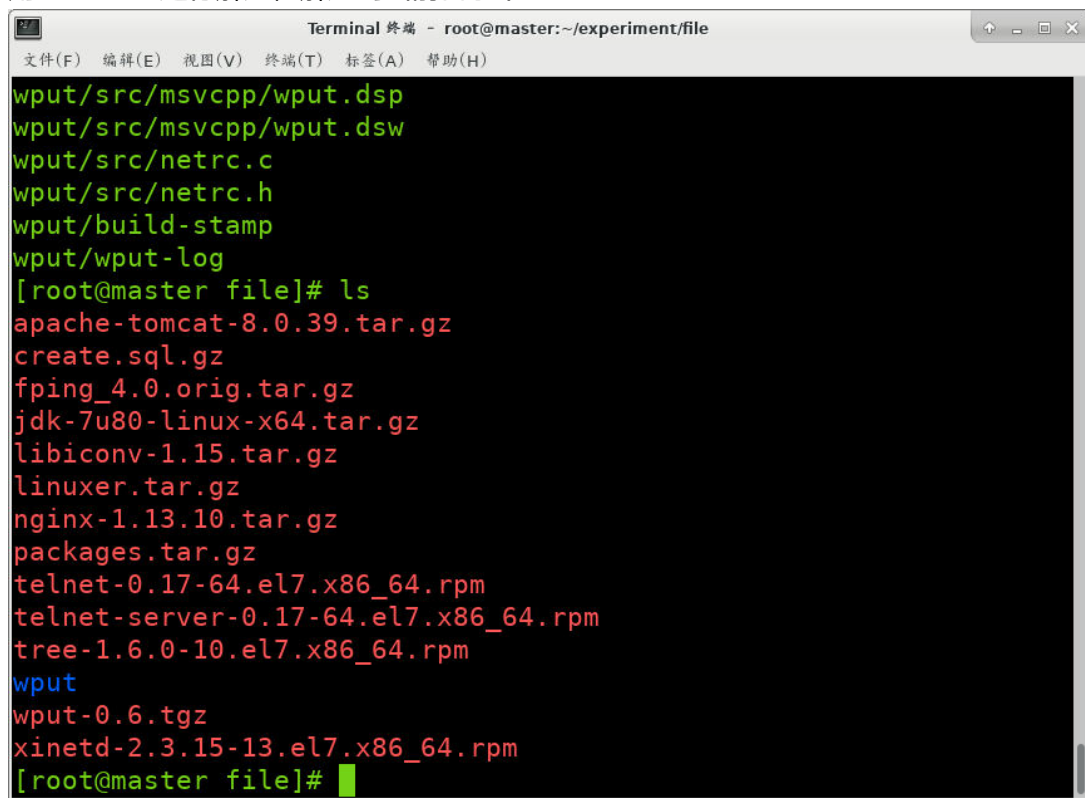
安装 jupyter notebook 插件 bash kernel, Bash Kernel 是一个允许你在 Jupyter Notebook 中运行 Bash 命令的程序。它使得 Jupyter Notebook 不仅限于执行 Python 代码, 还可以执行 shell 脚本和命令。这对于系统管理、文本处理、文件操作等任务非常有用。

Wput 安装

A terminal window titled "Terminal 终端 - root@master:~/experiment/file" showing the process of extracting a tar.gz file. The user lists several files in the directory, including linuxer.tar.gz, nginx-1.13.10.tar.gz, packages.tar.gz, telnet-0.17-64.el7.x86_64.rpm, telnet-server-0.17-64.el7.x86_64.rpm, tree-1.6.0-10.el7.x86_64.rpm, wput-0.6.tgz, xinetd-2.3.15-13.el7.x86_64.rpm, and then runs the command "tar -xvzf wput-0.6.tgz". The output shows the contents of the archive being listed.

```
linuxer.tar.gz
nginx-1.13.10.tar.gz
packages.tar.gz
telnet-0.17-64.el7.x86_64.rpm
telnet-server-0.17-64.el7.x86_64.rpm
tree-1.6.0-10.el7.x86_64.rpm
wput-0.6.tgz
xinetd-2.3.15-13.el7.x86_64.rpm
[root@master file]# tar -xvzf wput-0.6.tgz
wput/
wput/install-sh
wput/ABOUT-NLS
wput/COPYING
wput/ChangeLog
wput/INSTALL
wput/Makefile
wput/Makefile.in
wput/TODO
wput/aclocal.m4
wput/config.guess
wput/config.sub
wput/config.status
```

定位到/root/experiment/file/下，ls 查看目录，发现 wput-0.6.tgz （tar.gz 包），利用 tar -xvzf 进行解压，解压到当前目录下。

A terminal window titled "Terminal 终端 - root@master:~/experiment/file" showing the output of the "ls" command. The output lists the files in the directory, including the newly extracted wput directory and various tar.gz and rpm files.

```
wput/src/msvcpp/wput.dsp
wput/src/msvcpp/wput.dsw
wput/src/netrc.c
wput/src/netrc.h
wput/build-stamp
wput/wput-log
[root@master file]# ls
apache-tomcat-8.0.39.tar.gz
create.sql.gz
fping_4.0.orig.tar.gz
jdk-7u80-linux-x64.tar.gz
libiconv-1.15.tar.gz
linuxer.tar.gz
nginx-1.13.10.tar.gz
packages.tar.gz
telnet-0.17-64.el7.x86_64.rpm
telnet-server-0.17-64.el7.x86_64.rpm
tree-1.6.0-10.el7.x86_64.rpm
wput
wput-0.6.tgz
xinetd-2.3.15-13.el7.x86_64.rpm
[root@master file]#
```

```
Terminal 终端 - root@master:~/experiment/file/wput
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)
make[1]: *** [wput.o] 错误 1
make[1]: 离开目录“/root/experiment/file/wput/src”
make: *** [all] 错误 2
[root@master wput]# ./configure
Debug Information enabled
checking for gcc... gcc
checking for C compiler default output file name... a.out
checking whether the C compiler works... yes
checking whether we are cross compiling... no
checking for suffix of executables...
checking for suffix of object files... o
checking whether we are using the GNU C compiler... yes
checking whether gcc accepts -g... yes
checking for gcc option to accept ANSI C... none needed
checking how to run the C preprocessor... gcc -E
checking for egrep... grep -E
checking for ANSI C header files... yes
checking for sys/types.h... yes
checking for sys/stat.h... yes
checking for stdlib.h... yes
checking for string.h... yes
checking for memory.h... yes
```

进入解压后的 wput 文件夹内，进行编译./configure

```
Terminal 终端 - root@master:~/experiment/file/wput
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)
config.status: creating po/POTFILES
config.status: creating po/Makefile
[root@master wput]# make install
cd po && make all
make[1]: 进入目录“/root/experiment/file/wput/po”
make[1]: 对“all”无需做任何事。
make[1]: 离开目录“/root/experiment/file/wput/po”
cd src && make all
make[1]: 进入目录“/root/experiment/file/wput/src”
gcc -Wall -g -DLOCALEDIR=\"/usr/local/share/locale\" -c -o wput.o wput.c
wput.c: 在函数‘readwputrc’中:
wput.c:546:11: 警告: 将一个整数转换为大小不同的指针 [-Wint-to-pointer-cast]
    tmp = (char *) set_option(com, val);
           ^
wput.c: 在函数‘commandlineoptions’中:
wput.c:638:17: 警告: 格式‘%d’ expects argument of type‘int’, but argument 3 has type‘long unsigned int’ [-Wformat=]
    fprintf(opt.output, "\\nUsing %d-Bytes for off_t\\n", sizeof(off_t));
                    ^
```

然后 make install 进行安装

```
Terminal 终端 - root@master:~/experiment/file/wput
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)

[root@master wput]# /usr/sbin/wput
bash: /usr/sbin/wput: 没有那个文件或目录
[root@master wput]# wput
Nothing done. Try `wput --help'.
[root@master wput]# wput --help
Usage: wput [options] [file]... [url]...
      url      ftp://[username[:password]@]hostname[:port]/[path/]
               [file]]

Startup:
  -V, --version      Display the version of wput and exit.
  -h, --help         Print this help-screen
  -b, --background  go to background after startup
```

安装成功

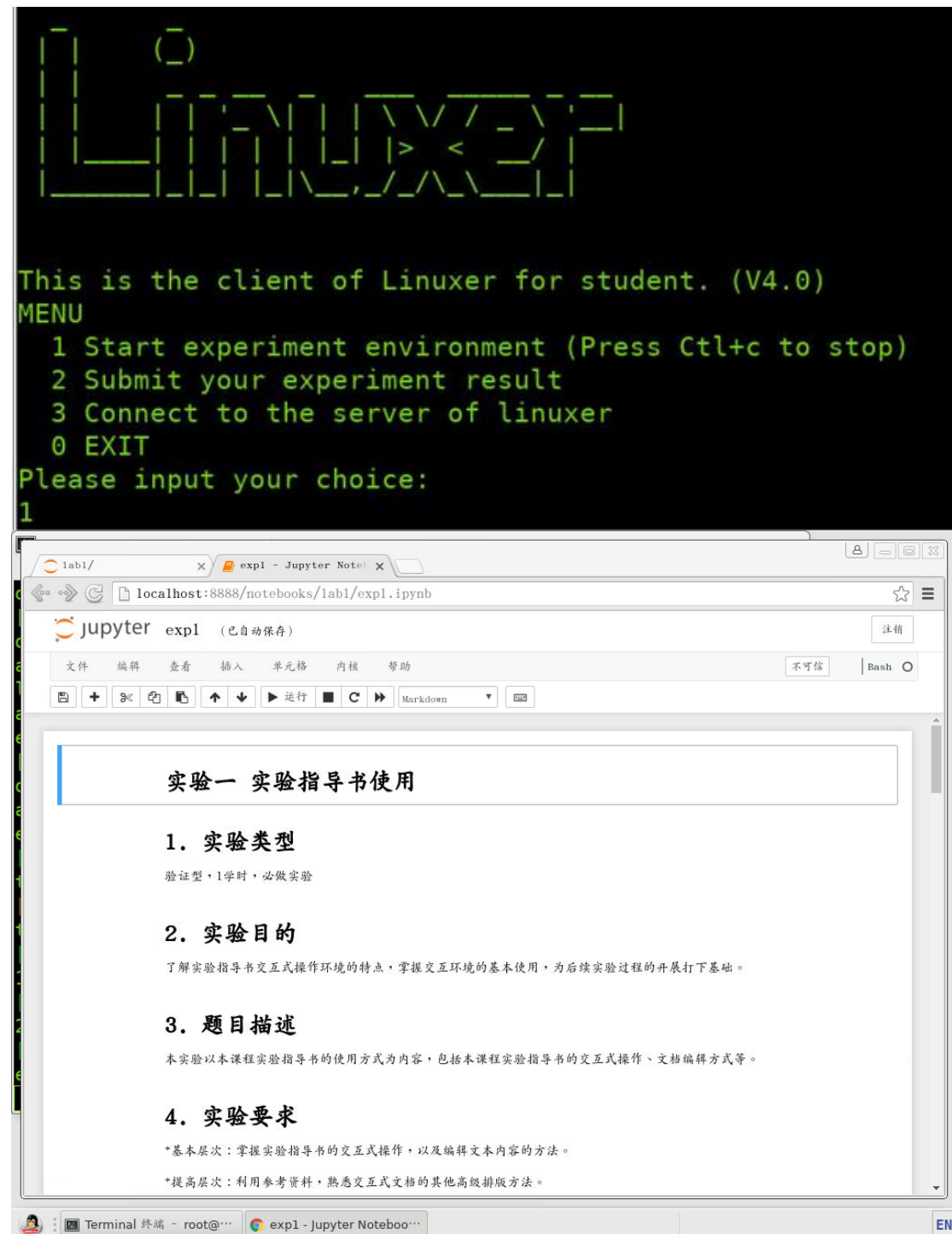
Linuxcer 安装

```
Terminal 终端 - root@master:~/experiment/file
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)

http://sourceforge.net/tracker/?group_id=141519
[root@master wput]# cd /root/experiment/file/
[root@master file]# ls
apache-tomcat-8.0.39.tar.gz
create.sql.gz
fping_4.0.orig.tar.gz
jdk-7u80-linux-x64.tar.gz
libiconv-1.15.tar.gz
linuxer.tar.gz
nginx-1.13.10.tar.gz
packages.tar.gz
telnet-0.17-64.el7.x86_64.rpm
telnet-server-0.17-64.el7.x86_64.rpm
tree-1.6.0-10.el7.x86_64.rpm
wput
wput-0.6.tgz
xinetd-2.3.15-13.el7.x86_64.rpm
[root@master file]# tar -xvzf linuxer.tar.gz
linuxer/
linuxer/lib/
linuxer/lib/server.lib
linuxer/lib/client.lib
```

与wput安装相同,先进入/root/experiment/file下查看文件目录,发现linuxer.tar.gz,使用tar -xvzf 进行解压,得到linuxer文件夹。

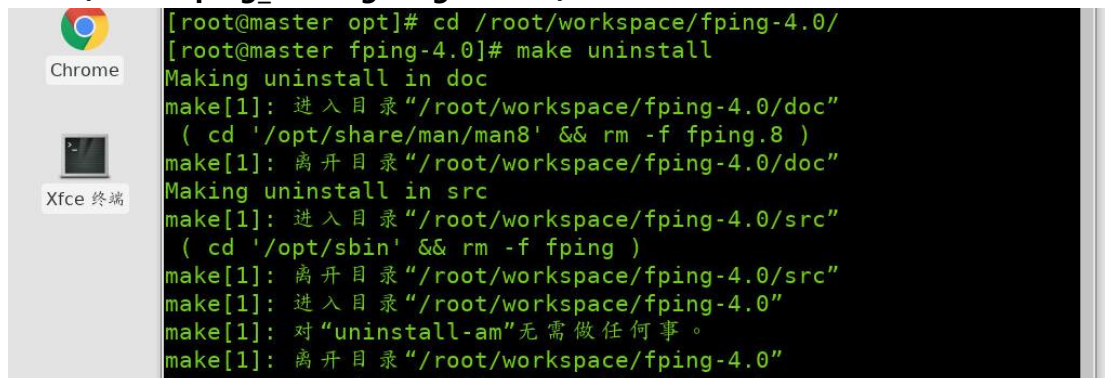
如果尝试启动刚刚安装的客户端应用程序 (linuxer)，但没有先运行 make 来编译源代码，那么可能会遇到找不到可执行文件或依赖项缺失的问题。



输入 1，启动实验环境，查看实验指导书。

四、实验思考或体会

实验一思考：如何完成.tar.gz 的卸载？能使用包管理器 RPM 查询已安装过的.tar.gz 吗？（以前述 fping_4.0.orig.tar.gz 为例子）



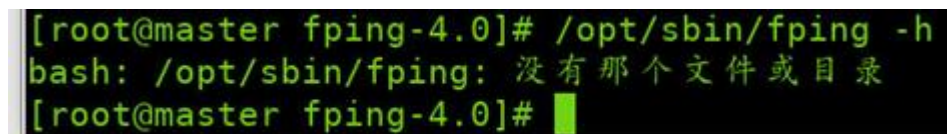
```
[root@master opt]# cd /root/workspace/fping-4.0/
[root@master fping-4.0]# make uninstall
Making uninstall in doc
make[1]: 进入目录"/root/workspace/fping-4.0/doc"
( cd '/opt/share/man/man8' && rm -f fping.8 )
make[1]: 离开目录"/root/workspace/fping-4.0/doc"
Making uninstall in src
make[1]: 进入目录"/root/workspace/fping-4.0/src"
( cd '/opt/sbin' && rm -f fping )
make[1]: 离开目录"/root/workspace/fping-4.0/src"
make[1]: 进入目录"/root/workspace/fping-4.0"
make[1]: 对"uninstall-am"无需做任何事。
make[1]: 离开目录"/root/workspace/fping-4.0"
```

进入 fping_4.0 文件夹下，输入 make uninstall 进行卸载。

make uninstall 命令会调用 Makefile 中定义的 uninstall 目标，执行卸载过程。Makefile 是一个包含有关于如何编译和安装（以及卸载）软件的指令的文件。

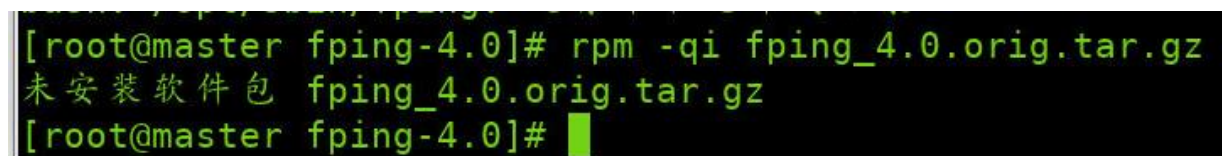
卸载步骤通常包括删除之前通过 make install 安装的文件。比如之前 make install 将可执行文件安装到了 /opt/sbin/fping，那么 make uninstall 将会从这个目录中删除这些文件。

在卸载完成后，可以使用 make clean 来删除编译过程中成的临时文件。



```
[root@master fping-4.0]# /opt/sbin/fping -h
bash: /opt/sbin/fping: 没有那个文件或目录
[root@master fping-4.0]#
```

如果卸载成功，则输入 /opt/sbin/fping -h 没信息返回。如上图，卸载成功。



```
[root@master fping-4.0]# rpm -qi fping_4.0.orig.tar.gz
未安装软件包 fping_4.0.orig.tar.gz
[root@master fping-4.0]#
```

rpm 是一个软件包管理系统，它允许用户在 Linux 系统上安装、卸载、查询和验证软件包。rpm 包通常用于 Red Hat 系列的 Linux 发行版，如 Red Hat Enterprise Linux、CentOS 和 Fedora。

对于 tar.gz 文件，rpm 包管理器通常不会直接管理这些文件，因为它们是源代码包或由 tar 命令创建的归档文件，而不是 rpm 包，tar.gz 文件通常需要手动解压、配置和编译。

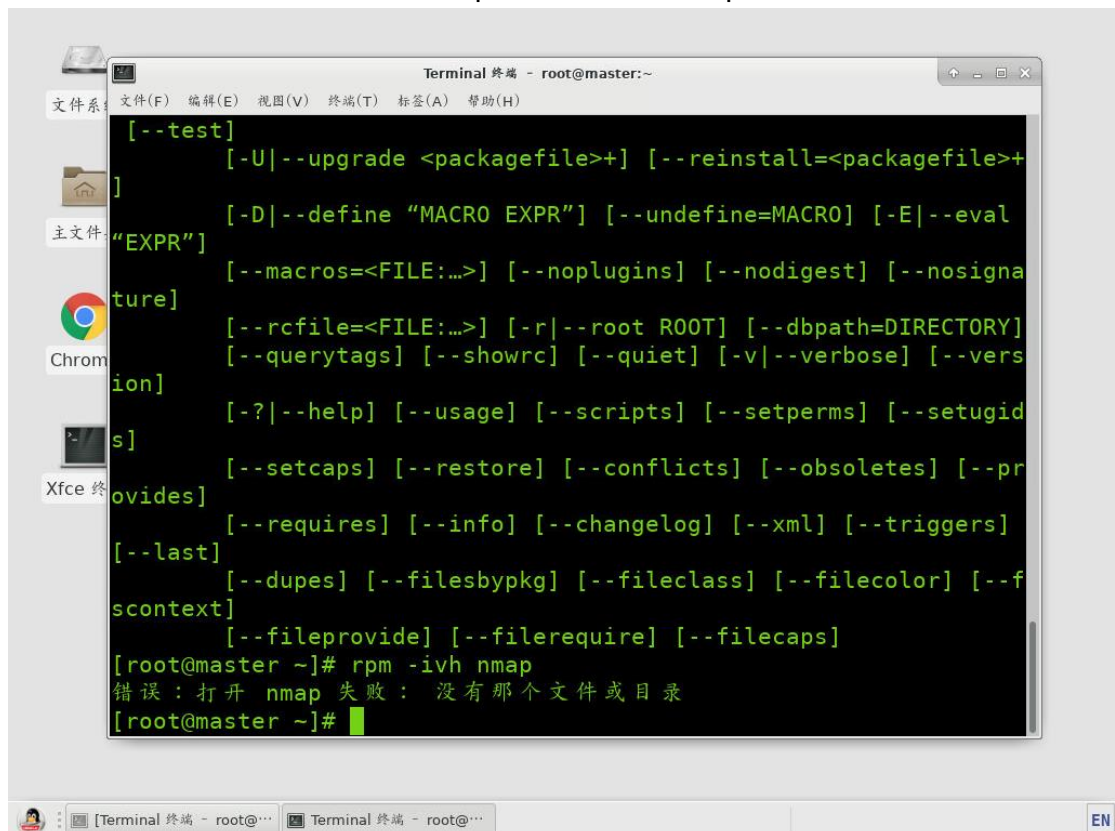
如果你想查询是否安装了一个特定的软件，你可以使用 rpm 来查询特定于 rpm 包的信息。rpm -qi fping_4.0.orig.tar.gz。使用该命令发现系统回显未安装。

这是因为 rpm 不会对 tar.gz 文件起作用，因为它们不是通过 rpm 安装的。如果`fping\_4.0.orig.tar.gz`被解压和手动安装，rpm 不知道它的存在。

实验二思考：实验环境中，rpm 软件和 yum 软件有什么关系？试举例说明。

yum 通常用来管理 rpm 包，它使用 rpm 作为后端来安装、删除和查询软件包。yum

可以处理软件包之间的依赖关系，而 rpm 不行，如下图。yum 需要仓库元数据来知道哪些软件包可用以及它们的依赖关系，而 rpm 命令直接操作 .rpm 文件。

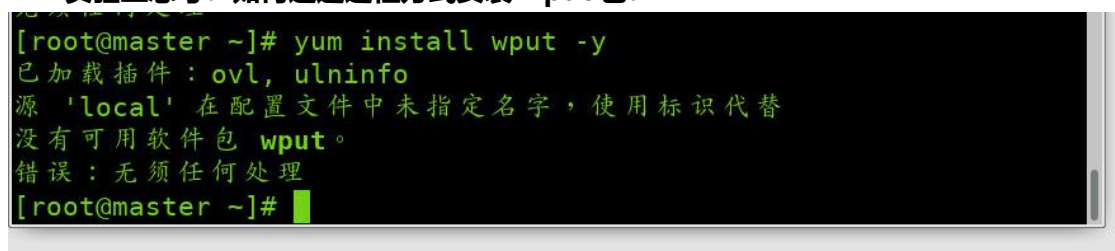


```
Terminal 终端 - root@master:~
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)

[--test]
[--U|--upgrade <packagefile>+] [--reinstall=<packagefile>+]
]
[--D|--define "MACRO EXPR" ] [--undefine=MACRO] [-E|--eval
"EXPR"]
[--macros=<FILE:...>] [--nopugins] [--nodigest] [--nosigna
ture]
[--rcfile=<FILE:...>] [-r|--root ROOT] [--dbpath=DIRECTORY]
[--querytags] [--showrc] [--quiet] [-v|--verbose] [--vers
ion]
[-?|--help] [--usage] [--scripts] [--setperms] [--setugid
s]
[--setcaps] [--restore] [--conflicts] [--obsoletes] [--pr
ovides]
[--requires] [--info] [--changelog] [--xml] [--triggers]
[--last]
[--duplicates] [--filesbypkg] [--fileclass] [--filecolor] [--f
scontext]
[--fileprovide] [--filerequire] [--filecaps]
[root@master ~]# rpm -ivh nmap
错误：打开 nmap 失败： 没有那个文件或目录
[root@master ~]#
```

如果使用 rpm 来安装 nmap,则需要先找到 nmap 的 rpm 包,然后使用命令 rpm -ivh nmap 进行安装，并且 nmap 相关依赖项也需要手动找到并安装。

实验三思考：如何通过远程方式安装 wput 包？



```
[root@master ~]# yum install wput -y
已加载插件：ovl, ulninfo
源 'local' 在配置文件中未指定名字，使用标识代替
没有可用软件包 wput。
错误：无须任何处理
[root@master ~]#
```

使用 yum 安装 wput, 报错“没有可用软件包”，这是因为软件源列表中没有包含 wput 的仓库。

尝试配置最新阿里云源，安装 wput。

```
Terminal 终端 - root@master:/etc/yum.repos.d
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)

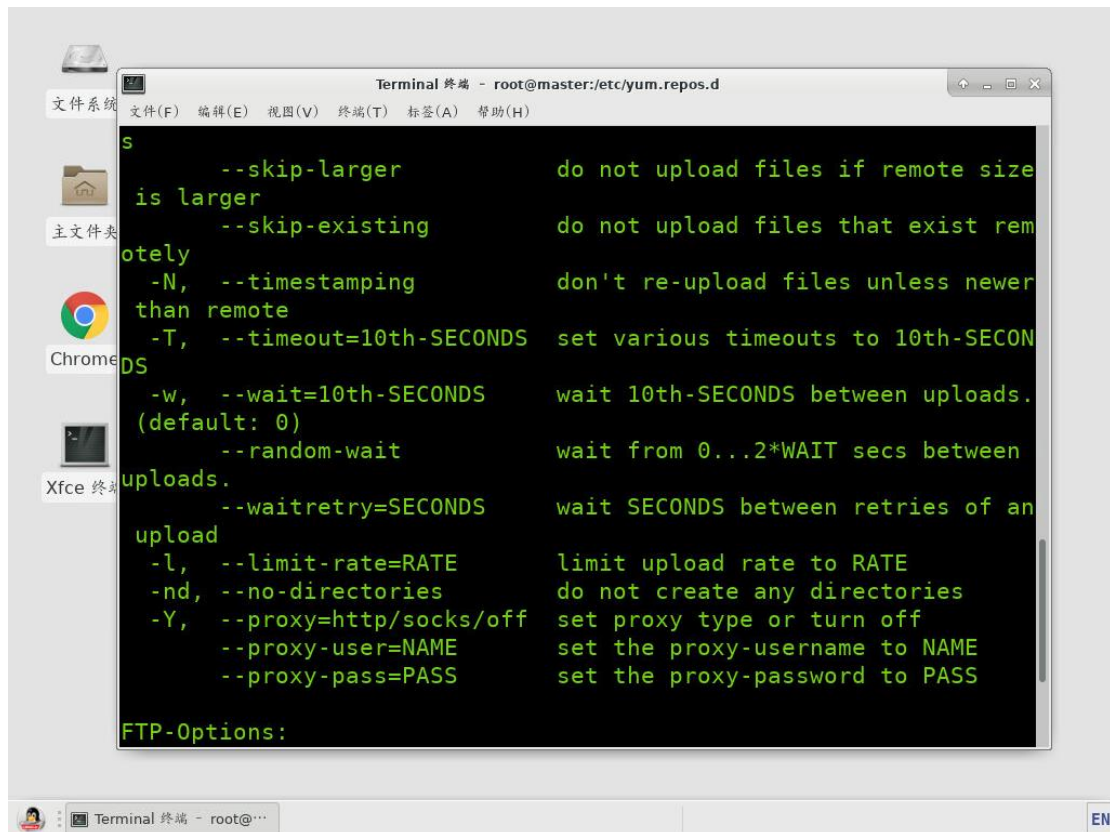
[ali_baseos]
name=ali_baseos
baseurl=https://mirrors.aliyun.com/centos-stream/9-stream/BaseOS/
x86_64/os/
gpgcheck=0

[ali_appstream]
name=ali_appstream
baseurl=https://mirrors.aliyun.com/centos-stream/9-stream/AppStream/
x86_64/os/
gpgcheck=0

Terminal 终端 - root@master:/etc/yum.repos.d
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)

[root@master ~]# cd /etc/yum.repos.d/
[root@master yum.repos.d]# view /etc/yum.repos.d/aliyun_yum.repo
[root@master yum.repos.d]# yum makecache
已加载插件：ovl, ulninfo
ali_appstream           | 4.4 kB    00:00
ali_baseos              | 3.9 kB    00:00
local                   | 2.9 kB    00:00
(1/11): ali_appstream/group | 101 kB    00:00
(2/11): ali_appstream/filelists_db | 9.6 MB    00:00
(3/11): ali_appstream/primary_db | 6.2 MB    00:00
(4/11): ali_appstream/other_db | 2.7 MB    00:00
(5/11): ali_appstream/modules | 8.2 kB    00:00
(6/11): ali_appstream/group_xz | 11 kB     00:00
(7/11): ali_baseos/filelists_db | 1.8 MB    00:00
(8/11): ali_baseos/primary_db | 5.9 MB    00:00
(9/11): ali_baseos/other_db | 802 kB    00:00
(10/11): ali_baseos/group_xz | 4.5 kB    00:00
(11/11): ali_baseos/group | 38 kB     00:05
元数据缓存已建立
[root@master yum.repos.d]#
```

安装也无效。



```
Terminal 终端 - root@master:/etc/yum.repos.d
--skip-larger          do not upload files if remote size
is larger
--skip-existing        do not upload files that exist rem
otely
-N, --timestamping     don't re-upload files unless newer
than remote
-T, --timeout=10th-SECONDS set various timeouts to 10th-SECON
DS
-w, --wait=10th-SECONDS wait 10th-SECONDS between uploads.
(default: 0)
--random-wait          wait from 0...2*WAIT secs between
uploads.
--waitretry=SECONDS   wait SECONDS between retries of an
upload
-l, --limit-rate=RATE  limit upload rate to RATE
-nd, --no-directories do not create any directories
-Y, --proxy=http/socks/off set proxy type or turn off
--proxy-user=NAME      set the proxy-username to NAME
--proxy-pass=PASS      set the proxy-password to PASS

FTP-Options:
```

根据 `wput -help` 发现可能可以使用 ftp 远程安装 wput 包，但需要先找到一个提供 wput 安装包的 ftp 服务器，然后使用 lftp 连接到 ftp 服务器，登陆下载 wput 安装包。因此 wput 未远程安装成功。

在未找到适合的 ftp 服务器下，wput 不能远程安装。

体会：

在本次实验中，我们探索了 Linux 系统上软件安装的多种方式，包括本地安装二进制包、通过网络使用包管理器安装软件，以及本地安装源代码包。

在实验一中，我们通过 tar.gz 包在 Linux 字符界面中进行了软件的本地安装。这个过程涉及到解压档案文件、编译源代码以及安装软件到指定路径，我们学会了如何通过阅读文档来更改默认的安装路径，以适应不同的系统配置或管理需求。本地安装提供了更多的灵活性，但需要更多地关注软件的依赖性。

在实验二中，我们通过网络使用 `yum` 包管理器远程安装了 `nmap`。这个实验让我们熟悉了如何配置和使用 Linux 的包管理器来安装、更新和卸载软件。`yum` 自动处理软件依赖性，这大大简化了安装过程。但安装软件前配置正确的网络和软件源是必不可少的。

实验三要求我们先远程安装 `jupyter notebook`，然后本地安装 `wput`，最后完成 `Linuxer` 客户端的安装。在安装过程中遇到的问题迫使我们去寻找解决方案，这增强了我们的问题解决能力，让我们学会了如何将不同来源的软件包整合到一个协调一致的系统中。

通过这些实验，我们不仅掌握了 Linux 字符界面中安装和卸载软件的方法，还了解了在不同情况下选择最合适安装方式的重要性。我们体会到了包管理器在依赖性管理和软件源配置方面的优势，同时也认识到了本地安装在定制化和灵活性方面的好处。